ILP-Based Scheduling for High-Level Synthesis

Milestone 1: Technical Understanding and Research Framing

Name	MD Rafiul Islam
Student ID	1232714
Course area	SS2026 ELE HW/SW Codesign Seminar
Topic	ILP-Based Scheduling for High-Level Synthesis
Document type	Structured technical briefing and preparation notes

Purpose: This document is a structured milestone artifact for understanding, analysis, evaluation, and scientific communication. It is written as technical notes, not as copied final prose.

1. Milestone 1 goal

Milestone 1 focuses on technical understanding and research framing. The goal is not a polished final paper; the goal is to show that the assigned topic is understood before final writing starts.

- Problem setting: scheduling operations in High-Level Synthesis while respecting data dependencies and limited hardware resources.
- Core technical idea: encode operation-to-cycle decisions as binary variables and solve a constrained optimization problem.
- Application example: y = a*b + c*d with two multiplications and one addition.
- Research framing: ILP provides exactness for the modeled objective, but the model size can grow quickly.

2. Problem statement and motivation

High-Level Synthesis starts from a behavioral algorithm and produces RTL hardware. Inside this flow, scheduling decides in which clock cycle each operation is executed. A schedule is valid only if operations receive their input values in time and the available functional units are not overused.

The main engineering tradeoff is latency versus hardware cost. More parallel functional units may reduce the number of clock cycles, but they increase area and resource usage. Fewer units save hardware, but the design may need more cycles.

Term	Meaning for ILP-based HLS scheduling
Operation node	An arithmetic or logical action, for example multiplication or addition.
Dependency edge	A rule saying that one operation must finish before another can start.
Resource constraint	A limit on how many operations of the same type can run in one cycle.
Latency	The number of cycles until the final result is available.
Objective	The optimization target, here mainly minimum latency.

3. Core example and operation graph

The example expression is y = a*b + c*d. It contains three operations. O1 computes m1 = a*b. O2 computes m2 = c*d. O3 computes y = m1 + m2. O3 depends on both O1 and O2, but O1 and O2 are independent.

```
O1: m1 = a * b
O2: m2 = c * d
O3: y  = m1 + m2
Dependencies: O1 -> O3, O2 -> O3
```

This small graph is enough to show the important scheduling decision. With one multiplier, O1 and O2 cannot run together. With two multipliers, they can run in the same cycle and the schedule becomes shorter.

4. Formal ILP model in simple form

The scheduling problem can be represented by a directed graph G = (V, E). V is the set of operations and E is the set of dependency edges. Each operation has a type such as MUL or ADD. Each resource type has an available count A_r.

```
Decision variable:
  x_i,t = 1 if operation i starts in cycle t, otherwise 0

Uniqueness:
  sum_t x_i,t = 1        for every operation i

Start time:
  S_i = sum_t t * x_i,t

Dependency:
  S_j >= S_i + delay_i     for every edge i -> j

Resource limit:
  number of operations of type r in cycle t <= A_r

Objective:
  minimize latency L
```

Important understanding point: ILP does not just "guess" a schedule. It defines all legal scheduling choices as variables and constraints, then selects the best legal solution for the chosen objective.

5. Algorithm and implementation framing

A full HLS tool would normally build an ILP model and pass it to a solver. The seminar implementation uses an exhaustive ILP-style search to keep the logic visible in C++.

- Create the operation list and dependency list.
- Choose resource counts, for example one or two multipliers.
- Enumerate possible operation-to-cycle assignments.
- Reject candidates that violate uniqueness, dependencies, or resource limits.
- Compute latency for feasible candidates.
- Keep the feasible schedule with the smallest latency.

Configuration	Expected schedule	Latency meaning
1 multiplier + 1 adder	Cycle 1: O1, Cycle 2: O2, Cycle 3: O3	Multiplications are serialized, so the result needs 3 cycles.
2 multipliers + 1 adder	Cycle 1: O1 and O2, Cycle 2: O3	Independent multiplications run in parallel, so the result needs 2 cycles.

6. Assumptions, limitations, and open questions

- Assumption: each operation has a simple known delay in cycles.
- Assumption: resource types are simple, for example MUL and ADD.
- Limitation: the example does not cover loops, memory dependencies, pipelined datapaths, or branches.
- Limitation: exhaustive search is easy to explain but does not scale to large graphs.
- Open question: Should the final paper mention professional solvers such as CPLEX or Gurobi only as context, or should the implementation stay strictly solver-free?
- Open question: Should RTL realization be described as an HLS output example or as a manual validation of the schedule?

7. Source-use notes

Reference	How it supports this topic
Teich and Haubelt, 2007	German HLS / HW/SW synthesis background: allocation, scheduling, binding, optimization.
De Micheli, 1994	Main textbook-level foundation for synthesis and optimization of digital circuits.
McFarland, Parker, Camposano, 1990	Classic overview of high-level synthesis and behavioral synthesis flow.
Paulin and Knight, 1989	Neighboring heuristic approach: force-directed scheduling for behavioral synthesis.
Cong et al., 2011	Modern FPGA-oriented HLS context and deployment relevance.
Rettberg, 2026 seminar slides	Course requirements: scientific paper, C/C++ feature, application example, RTL component realization.